

# Parallel Simulation for AI Training in *Crash Bandicoot 1*

## Introduction

Training an AI to beat a *Crash Bandicoot 1* speedrun involves combining **demonstration learning** with **reinforcement learning (RL)** and efficient use of emulation. The goal is to feed a “perfect run” (a predetermined sequence of controller inputs that completes a level quickly) into the system, and then have the AI improve upon it – i.e. **learn from the example and attempt to surpass the timing**. This is a challenging task, but recent advancements in game emulation and machine learning provide a roadmap. Key components include using an **emulator (or modded console)** to run the game, leveraging **Python-based AI frameworks** (such as OpenAI’s Gym/Stable-Baselines), and possibly running **parallel simulations** to speed up learning. Below, we break down the approach and important considerations, with comprehensive information on each aspect.

## Emulation vs. Modded Hardware for AI Control

To train an AI on *Crash Bandicoot*, an **emulator** is typically the practical choice. Emulators like **RetroArch** (with a PS1 core) or others (e.g. PCSX, BizHawk) can run the game on a PC and allow programmatic control. In contrast, using a **modded PS1 console** would require hardware interfaces to inject inputs and read game state, which is far more complex and slow. Emulators offer several crucial advantages:

- **Deterministic, Controllable Environment:** Emulators can run the game deterministically (given the same input sequence, the same outputs occur) and even allow saving/loading state. This determinism enables recording a sequence of inputs (the demo run) and replaying it exactly for training or testing <sup>1</sup>. It also means an AI agent can practice the same scenario repeatedly under identical conditions – impossible on real hardware without manual resets. Emulators also support fast-forwarding or removing frame limits, speeding up simulations. All these are essential for AI training which may require millions of frames of gameplay.
- **Programmatic Input and Data Access:** Through emulator APIs or scripting, one can **inject controller inputs** frame-by-frame from Python or other languages. For example, the **OpenAI Gym Retro** platform (specifically its maintained fork **Stable-Retro**) wraps emulators via the libretro API, turning games into environments where you can step the game with custom inputs <sup>2</sup>. Each game integration defines memory addresses for game variables (score, lives, timers, etc.), reward functions, and episode end conditions <sup>2</sup>. In the case of *Crash Bandicoot*, one would integrate the PS1 ROM, specifying how to detect level completion and measure performance. This setup allows the AI to **read game state** (e.g. Crash’s position, level timer if any, or simply know when the level is finished) and systematically **apply actions** each frame.
- **Parallel instances and automation:** On a PC, you can run multiple instances or threads of the emulator for parallel simulation (discussed more below), and use automation tools. By contrast, a

single physical console can only run one game instance at real-time speed. Emulation thus vastly increases the data throughput for learning.

**In summary**, while a *modded PS1* could theoretically be used to *demonstrate* the final learned controller sequence (for example, by feeding the AI's inputs into the console to showcase a perfect play on real hardware, as done in some tool-assisted speedruns), all the **training and experimentation are best done in an emulator environment** for speed and convenience <sup>1</sup>. Emulators like RetroArch (with libretro) are well suited, and frameworks exist to integrate them with Python for AI research.

## Feeding a Predetermined “Perfect Run” (Demonstration)

A core idea is to provide the AI with a **reference optimal playthrough** – essentially a sequence of inputs performing a near-perfect run of a level – and then let the AI learn from this *demonstration*. There are a few ways to utilize this demonstration data:

- **Recording and Playback:** Using the emulator, you can record the button inputs of the perfect run. For instance, Stable-Retro (Gym Retro fork) allows creating a `.bk2` **movie file** which stores the initial game state and the sequence of button presses <sup>1</sup>. Because the emulator is deterministic, playing back this file reproduces the exact run every time <sup>1</sup>. The recorded run can serve as ground truth data. It's similar to how tool-assisted speedruns (TAS) are made by recording frame-perfect inputs <sup>1</sup>. You can play this back to measure its completion time and also supply it to the training process.
- **Imitation Learning (Behavioral Cloning):** One approach is to train a model to **mimic the demonstration**. For example, one could use **behavioral cloning**, treating the recorded states (game screens or memory states) and actions (controller inputs) from the perfect run as training data. A neural network (e.g. a Convolutional Neural Network taking the screen image) can be trained to predict the next action given the current state <sup>3</sup>. In fact, a community project “*pybandicoot*” took this approach: it collected thousands of frames of *Crash Bandicoot* gameplay with corresponding keystrokes and trained a CNN (inspired by a self-driving car game demo by Sentdex) to play the first level <sup>3</sup> <sup>4</sup>. The imitation model got surprisingly far (reportedly ~75% through the level on its own) by following the patterns it learned <sup>4</sup>. This shows that learning from demonstration is feasible. However, pure imitation has limitations – the agent might fail if it strays off the demonstrated trajectory, since it hasn't learned how to recover or try new strategies.
- **Using Demonstrations in Reinforcement Learning:** A more powerful technique is to combine the demonstration with RL, so the agent can **surpass the demonstrator**. The demonstration provides a strong prior, and then the agent continues to improve via trial-and-error. Research has shown that an RL agent can indeed exceed expert performance by learning from demo data then exploring further <sup>5</sup>. For instance, *Normalized Actor-Critic (NAC)* is an algorithm that first learns from demonstrations and then “*refines the policy in the environment, surpassing the demonstrator's performance*” <sup>5</sup>. In practical terms for *Crash Bandicoot*: you could initialize the agent's policy using the perfect run data (so it knows efficient movement patterns), and then let it **experiment with variations** in a simulated environment to find even faster routes or glitch exploits that a human might have missed. The reward function would be designed to encourage finishing the level faster than the recorded run. For example, if the demonstrator's time is  $T$  frames, the agent gets a positive reward for finishing, with a bonus for doing so in fewer than  $T$  frames (and perhaps a larger bonus the faster it is). A simpler

reward shaping is to give a small negative reward each time-step (to encourage speed) and a large positive reward at level completion <sup>5</sup>. The agent will try to minimize time to maximize reward.

- **Altering the Input Sequence:** As the agent learns, it will start modifying the sequence of inputs. Initially it might follow the demonstration closely (especially if using imitation learning). Over time, with exploration, it may discover modifications: e.g. *taking a tighter jump, skipping a crate, or using a glitch* that cuts down time. This process is essentially **trial-and-error search** guided by the reward of finishing faster. In an evolutionary context, you could even treat the input sequence as a genome and use a **genetic algorithm** to evolve faster runs. In fact, one hobbyist on Reddit attempted a *Crash 1* AI using a genetic algorithm, letting multiple instances play and mutate over time <sup>6</sup>. The AI did gradually improve (it was even streamed live as it learned), though genetic approaches can be slow to converge <sup>6</sup>. More commonly, deep RL (with algorithms like PPO or DQN) plus demonstrations is used for efficiency.

In summary, **the demonstration is a valuable starting point** – it can drastically cut down learning time by preventing the agent from flailing randomly at the start. From that base, the agent uses *reinforcement learning* to try new actions and shorten the completion time, effectively *racing against the ghost* of the provided run.

## Integrating Python and OpenAI Tools

To implement the above, you'll make heavy use of Python-based AI libraries and possibly OpenAI's toolkits:

- **OpenAI Gym/Stable-Retro Environment:** Using the Gym interface for the game is highly recommended. With *Stable-Retro* (a fork of OpenAI Gym Retro maintained by the Farama foundation), you can integrate Crash Bandicoot as a custom environment. This involves providing the PS1 ROM (legally obtained) and creating an integration (with the Integration UI or via a JSON/spec files) that tells the framework how to identify key memory addresses (for rewards and termination) <sup>2</sup>. Once integrated, you can do `env = retro.make(game="CrashBandicoot-PS1", state="Level1")` in Python, and you'll have a **Gym environment** where calling `env.reset()` starts the level (using a savestate at the beginning), and `env.step(action)` advances the game by one frame with the given controller input. The observation can be the raw image or RAM state, and the `info` dict can include custom game variables (like player health or time) <sup>7</sup> <sup>8</sup>. This setup abstracts away a lot of low-level details and lets you plug the game into algorithms.
- **Stable-Baselines3 (SB3) or other RL libraries:** Stable-Baselines3 is a high-level library providing implementations of algorithms like PPO, DQN, etc., that work with Gym environments. You can connect SB3 to the Crash environment to train an agent. For example, one could use **PPO (Proximal Policy Optimization)**, which is a popular choice for continuous game environments. In fact, a recent project demonstrated exactly this: an AI learned to play *Crash Bandicoot* (the first level) using **Stable-Retro + Stable-Baselines (PPO)** as the tech stack <sup>9</sup>. The training involved vision (using the screen image via OpenCV) and a PyTorch neural network policy. Over training, the AI became capable of completing the level on its own. This shows that *Crash Bandicoot* is within reach for deep RL methods. Using Python, you can script the entire loop: reset environment, have the model choose an action, step the game, get reward, train the model – all in an automated fashion.

- **OpenAI APIs / Neural Network Models:** The mention of “OpenAI” by the user likely refers to the OpenAI Gym/Retro platform or possibly OpenAI’s research. (It’s not about using the OpenAI GPT API here, but rather their tools for RL.) The neural network (policy) that drives the agent will be a part of your Python code (e.g. a PyTorch or TensorFlow model). You might design a convolutional network to process the game frames, or use the **RAM state as input** if you want to avoid image complexity (Gym Retro can expose RAM arrays as observations too <sup>7</sup>, although for a 3D platformer like Crash, raw pixels might capture more nuance like depth). The output of the network could be a distribution over possible button combos. Note that controlling a PS1 game involves potentially **complex action space** (the controller has D-pad, buttons, etc.). You will likely want to **discretize** the action space – e.g., limit to meaningful combinations (like “run forward”, “run forward + jump”, “spin attack”, etc.). Gym Retro allows custom discretizers to map the many-button binary space into a simpler set of actions <sup>10</sup>.
- **Scripting and Tools:** Python can also be used for utility tasks: e.g., using `retro.record()` to record runs to file, or employing libraries like *pyautogui* to send key presses if using a simpler approach outside Gym. Some have scripted emulators by capturing the window and sending inputs (like the pyBandicoot project did with direct keyboard events <sup>4</sup>), but using Gym Retro is more robust and gives you structured data. You might also use OpenCV for computer vision tricks (e.g., to detect certain screen events if not using RAM).

In short, **Python** serves as the glue between the game and the AI algorithm. It orchestrates the emulator (via Gym Retro or other APIs) and runs the learning algorithm (PPO, DQN, genetic algorithm, etc.), possibly utilizing **OpenAI’s Gym interface** for ease of integration. The heavy-lifting of the AI (neural network computations, gradient updates) can be accelerated with libraries like PyTorch/TensorFlow using a GPU, while the game simulations run on the CPU.

## Parallel Simulation for Faster Learning

Training an AI for complex games can be very time-consuming if done naively. *Crash Bandicoot* is a 3D platformer, more complex than the Atari games traditionally tackled by RL, so efficiency is key. **Parallel simulation execution** refers to running multiple game instances simultaneously to speed up experience gathering. This technique is widely used in modern RL research to cut down training time <sup>11</sup> <sup>12</sup>:

- **Multiple Environments in Parallel:** Instead of training on one game instance step-by-step, you can launch, say, 8 or 16 parallel *Crash Bandicoot* environments and have the agent interact with all of them concurrently. Frameworks like Stable-Baselines3 make this easy by providing a *VecEnv* (vectorized environment) wrapper. For example, SB3 can use `SubprocVecEnv` to manage multiple processes each running a copy of the game, feeding experience into a central learner. The effect is that you collect 16 time-steps of experience in the time it normally takes to collect 1 (assuming you have enough CPU cores) <sup>12</sup>. OpenAI Baselines (the predecessor of SB3) even allowed launching parallel runs via MPI across processes <sup>13</sup>. This massively accelerates training because the AI sees more variations and situations per real-time second.
- **Faster Convergence:** With parallel environments, the training algorithm can update the model with a larger batch of diverse experiences at once. This often leads to more stable and faster convergence. In robotics RL research, using parallel simulation and large batch updates has cut training times dramatically. For instance, a recent paper “*FastTD3*” highlights that with techniques

like **parallel simulation** and large batch learning, complex control tasks can be learned in just a few hours instead of days <sup>11</sup>. While that example is about humanoid robots, the same principle applies to game AI: by leveraging multiple simulators running in parallel, the agent improves much more quickly than it would from a single stream of experience.

- **Distributed or Cloud Simulation:** If one wanted to go even further, you could distribute the workload across multiple machines or use cloud instances to run dozens of parallel game simulations. This is the approach DeepMind used for games like *Atari* (they had farm of machines running instances for DQN). However, for a single level of *Crash Bandicoot*, you likely won't need massive scale – a modern PC with multi-core CPU can run many instances. Keep in mind the PS1 emulator is more CPU-intensive than an Atari emulator, so parallelization might be limited by hardware. Still, even a few in parallel (and possibly frame-skipping techniques to skip redundant frames) will help.
- **Use in Evolutionary Algorithms:** If instead of gradient-based RL you explore a genetic algorithm or other search method, parallel simulation is also vital. A genetic algorithm would evaluate a *population* of runs in each generation. Those evaluations (playing the game with a given input sequence or AI policy) can be done in parallel on multiple emulator instances. This allows the algorithm to test many candidate solutions quickly before selecting and mutating for the next generation. Without parallelism, a GA trying to brute-force a long game level would be prohibitively slow.

Thus, **parallel simulation execution** is a key consideration – it “lets us train on  $n$  environments per step instead of just one” <sup>14</sup>, speeding up learning proportionally. It makes the difference between waiting days vs. hours for the AI to catch up to and surpass your perfect run.

## Additional Considerations and Information

- **State Representation & Rewards:** Defining what information the AI gets and what reward it receives is crucial. Options include raw pixels vs. RAM vs. high-level features. Pixels are general but require a deep CNN to interpret; RAM might offer direct access to player coordinates, etc., if one can locate those addresses. For reward, as mentioned, time to finish is the primary objective. If *Crash Bandicoot 1* levels don't have an internal timer, you can implicitly treat the frame count or steps taken as the “time”. One common reward shaping is **-1 per step and +100 (for example) at level completion**, so the agent is incentivized to finish in as few steps as possible. Also ensure the episode terminates when Crash dies or the level ends, so the agent can restart and learn from failures <sup>2</sup>.
- **Handling Complexity:** *Crash Bandicoot*'s mechanics (running, jumping, spinning, timing cycles of hazards) are quite complex compared to, say, an Atari game. The AI might require a fairly large neural network and a lot of training to master the level. Using the demonstration will give it a huge head-start, but expect some training time to fine-tune behaviors. If the perfect run involves very precise moves, the AI might initially struggle to replicate them exactly – it might find alternate ways that are safer until it gains the precision to do the fast strategy. Continuous monitoring of training is needed to ensure it's actually improving and not just doing random things or getting stuck.
- **Existing Projects and Resources:** The community interest in applying AI to retro games means you are not starting from zero. As noted, one GitHub project by **paulo101977** integrated PS1 games into

Gym Retro and specifically applied PPO to Crash Bandicoot <sup>9</sup>. His repositories (e.g., *CrashBandicootRL* and *my\_retro* for integration) could be insightful – they likely contain the integration files for Crash (memory addresses for crates, end-of-level, etc.) and training code. Additionally, the Stable-Retro documentation and Discord are great resources on how to add new games and set up training scenarios. There's also prior art in using **PyTorch models to override game input** via RetroArch's runahead features – for example, a RetroArch fork was made to allow a trained model to take control of player 2, etc., for AI vs AI matches <sup>15</sup>. This is more for applying a trained model in real-time, but it underscores that the pipeline from training to actual gameplay is feasible.

- **Using Real Hardware:** If after training your AI policy you want to see it run on a real PlayStation, it is possible but requires some creativity. One could use a device (like a microcontroller or Raspberry Pi with a PS1 controller interface) to **playback the learned input sequence** on a console. Essentially this turns the AI or its discovered sequence into a tool-assisted run on hardware. This has been done in other contexts (for instance, TASBot plays Tool-Assisted Speedruns on real consoles by feeding inputs). However, note that this would be just a replay of the sequence. If you wanted the AI model itself to *live-decide* actions on the console, you'd need a way to get the game's visuals or state from the console into your PC running the model in real-time – which is very hard without instrumentation. So, typically the model makes decisions in the emulated environment. But replaying a found optimal sequence on hardware could be a fun validation that “the AI beat the level faster than a human, on a real console.”
- **Ethical/Community Aspect:** Tool-assisted speedrun communities sometimes debate AI usage. A fully AI-discovered speedrun (especially if it finds new glitches) is an exciting development, but some TAS enthusiasts note that TAS runs are usually a collaborative, manual effort – an AI might “miss the point” of human ingenuity <sup>16</sup>. Regardless, from a research perspective, getting an AI to beat a human speedrun is an interesting challenge that combines imitation and innovation. If your AI finds a novel strategy, it could contribute to the speedrunning community's knowledge.

## Conclusion

Achieving parallel simulation execution for training an AI agent in *Crash Bandicoot 1* is a multi-faceted task. By using an **emulator (RetroArch/libretro)**, you can feed predetermined inputs (the *perfect run*) into the game and have a controlled, repeatable environment for the AI <sup>1</sup>. Leveraging **Python** with OpenAI's Gym/Stable-Retro interface and libraries like Stable-Baselines3, the AI can learn via **deep reinforcement learning**, starting from the demonstration and exploring improvements <sup>5</sup>. To accelerate learning and improve results, you run many simulations in parallel, utilizing modern compute to gather experience quickly <sup>11</sup> <sup>12</sup>. All necessary components – from recording replays to defining reward functions – must be set up, but tools and prior examples exist to guide you. With a careful integration of these pieces, the computer can indeed learn to **beat your best time** in Crash Bandicoot, potentially uncovering super-human optimizations along the way. The end result will be an AI-driven bandicoot blazing through the level even faster than the human “perfect” run!

### Sources:

- Stable-Retro documentation (Farama Foundation) – details on integrating games, recording replays, and deterministic emulator behavior <sup>1</sup> <sup>2</sup>.

- Reddit post by *AgeOfEmpires4AOE4* (Paulo) – notes that Crash Bandicoot was made into a Gym Retro environment and trained with Stable-Baselines PPO <sup>9</sup> .
- *Reinforcement Learning from Demonstrations* (Gao et al. 2018) – algorithm (NAC) that learns from imperfect demonstrations and surpasses expert performance <sup>5</sup> .
- *FastTD3* paper (2025) – highlights the efficacy of parallel simulation and large batches in speeding up RL training <sup>11</sup> .
- AurelianTactics, *Using Gym Retro with Baselines* – mentions how to run multiple environments in parallel for RL training <sup>12</sup> .
- Winded's *pybandicoot* project – example of using a CNN to play Crash Bandicoot via direct screen input and learned behavior <sup>3</sup> <sup>4</sup> .
- Reddit discussion on a Crash 1 genetic algorithm AI – anecdotal evidence of evolutionary approaches and the slow learning process without guided demonstrations <sup>6</sup> .

---

<sup>1</sup> <sup>7</sup> <sup>8</sup> <sup>10</sup> Python API - Stable-Retro Documentation

<https://stable-retro.farama.org/python/>

<sup>2</sup> GitHub - Farama-Foundation/stable-retro: Retro games for Reinforcement Learning

<https://github.com/Farama-Foundation/stable-retro>

<sup>3</sup> <sup>4</sup> GitHub - Winded/pybandicoot: Python plays Crash Bandicoot

<https://github.com/Winded/pybandicoot>

<sup>5</sup> [1802.05313] Reinforcement Learning from Imperfect Demonstrations

<https://arxiv.org/abs/1802.05313>

<sup>6</sup> AI that learns to play Crash 1 (genetic algorithm) : r/crashbandicoot

[https://www.reddit.com/r/crashbandicoot/comments/nbchv4/ai\\_that\\_learns\\_to\\_play\\_crash\\_1\\_genetic\\_algorithm/](https://www.reddit.com/r/crashbandicoot/comments/nbchv4/ai_that_learns_to_play_crash_1_genetic_algorithm/)

<sup>9</sup> <sup>15</sup> RetroArch fork with Machine Learning support : r/RetroArch

[https://www.reddit.com/r/RetroArch/comments/1hvtfl1/retroarch\\_fork\\_with\\_machine\\_learning\\_support/](https://www.reddit.com/r/RetroArch/comments/1hvtfl1/retroarch_fork_with_machine_learning_support/)

<sup>11</sup> [2505.22642] FastTD3: Simple, Fast, and Capable Reinforcement Learning for Humanoid Control

<https://arxiv.org/abs/2505.22642>

<sup>12</sup> <sup>13</sup> Retro Gym with Baselines: 4 Basic Usage Tips | by AurelianTactics | aureliantactics | Medium

<https://medium.com/aureliantactics/retro-gym-with-baselines-4-basic-usage-tips-1842d9aeff5>

<sup>14</sup> Vectorized Environments — Stable Baselines 2.10.3a0 documentation

[https://stable-baselines.readthedocs.io/en/master/guide/vec\\_envs.html](https://stable-baselines.readthedocs.io/en/master/guide/vec_envs.html)

<sup>16</sup> Why isn't machine learning used for TAS runs, or even in its ... - Reddit

[https://www.reddit.com/r/speedrun/comments/ocr9kb/why\\_isnt\\_machine\\_learning\\_used\\_for\\_tas\\_runs\\_or/](https://www.reddit.com/r/speedrun/comments/ocr9kb/why_isnt_machine_learning_used_for_tas_runs_or/)